LuminaTech

EcoGuia — Alma Digital Sustentável Sistema de avatares gamificados integrado à plataforma SoulUp para recompensa e exibição de ações ecológicas dos usuários.

INTEGRANTES DO GRUPO:

Ana Paula Cunha Brum - RM571359

Gabriella Serni Ponzetta - RM566296

Julio Cesar Iwata de Oliveira Barros - RM573723

Rafael Santos Dias - RM574105

Victor Felipe Silva Alencar – RM574057

São Paulo 2026

SUMÁRIO

1. Objetivo e Propósito Principal do Sistema 3
2. Escopo e Limitações do Projeto 4
3. Descrição das Funcionalidades 5
4. Tabela de Endpoints 6
5. Protótipo do Projeto 8
6. Diagrama de Classes 12
7. Procedimento para Rodar a Aplicação 17
8. Modelo de Entidade-Relacionamento (MER) 18
9. Script das Tabelas 19

OBJETIVO

O sistema é uma aplicação de gamificação, cujo propósito é incentivar e recompensar o engajamento dos usuários através de um avatar digital que evolui conforme o uso do sistema. Cada ação do usuário se transforma em progressão visível — experiência (XP) acumulada, subida de nível e conquista de emblemas (badges) — tornando o engajamento algo tangível e motivador.

O sistema permite que o usuário vincule um avatar personalizado, que acumula experiência e avança de nível conforme o engajamento evolui. Emblemas (badges), classificados como normais ou lendários, representam marcos e reconhecimentos dentro dessa trilha de progressão.

ESCOPO

O Sistema permite:

- Contempla o cadastro, consulta, atualização e exclusão (CRUD completo) das seguintes entidades:
- **Usuário** — gerenciamento dos dados de identificação, incluindo nome, e-mail, senha, cidade, vínculo com um avatar e ofensiva atual.
- **Avatar** — representação digital do usuário, contendo nome, nível atual e XP acumulado, com regras de ganho de experiência e progressão de nível.
- **Badge** — catálogo de emblemas disponíveis no sistema, contendo nome, tipo e descrição.
- **Conquista** — registro das conquistas dos usuários, estabelecendo o vínculo entre um usuário e um badge, além da data em que a conquista foi registrada.
- O sistema não contempla um catálogo de desafios ou atividades.
- A progressão do avatar é controlada por meio das regras de experiência (XP) implementadas na aplicação.
- O sistema contempla o registro do vínculo entre usuários e badges por meio da entidade Conquista.
- A persistência dos dados é realizada por meio de JDBC, utilizando banco de dados Oracle.
- A aplicação disponibiliza uma API RESTful desenvolvida com Spring Boot, permitindo a comunicação e integração com o front-end.

DESCRIÇÃO DAS FUNCIONALIDADES

Gestão de Usuários

- Cadastro de novos usuários, com vínculo obrigatório a um avatar.
- Consulta de todos os usuários ou de um usuário específico.
- Atualização de informações cadastrais (nome, e-mail, senha, cidade, avatar vinculado e ofensiva atual).
- Exclusão de usuários.

Gestão de Avatares

- Cadastro de avatares, com nome, nível inicial e XP atual.
- Consulta de todos os avatares ou de um avatar específico.
- Atualização dos dados do avatar.
- Progressão automática de nível durante a atualização, conforme a quantidade de XP acumulada: a cada 1.000 XP, o avatar avança um nível.
- Exclusão de avatares.

Gestão de Badges (Emblemas)

- Cadastro de badges, classificados como normal ou lendário.
- Consulta de todos os badges ou de um badge específico.
- Atualização dos dados de um badge.
- Exclusão de badges.
- Validação de integridade: o sistema aceita apenas os tipos de badge estabelecidos (normal/lendario), por meio de validação na aplicação.

Gestão de Conquistas

- Cadastro de conquistas, vinculando um usuário a um badge.
- Consulta de todas as conquistas ou de uma conquista específica.
- Atualização dos dados de uma conquista.
- Exclusão de conquistas.
- Registro automático da data atual quando a data da conquista não é informada no cadastro ou na atualização.

Interface de Integração (API RESTful)

- Disponibilização de endpoints REST para as operações de cadastro, consulta, atualização e exclusão das entidades Usuário, Avatar, Badge e Conquista.
- Utilização dos métodos HTTP GET, POST, PUT e DELETE para realização das operações.
- Recebimento e envio de dados no formato JSON, permitindo a integração com o front-end.
- Validação dos dados recebidos nas requisições, com utilização de Jakarta Bean Validation.
- Configuração de CORS para permitir a comunicação entre a API e o front-end.
- Retorno de respostas HTTP conforme o resultado das operações.

TABELAS DE ENDPOINTS

A tabela a seguir apresenta os endpoints disponibilizados pela API EcoGuia, organizados por recurso e método HTTP. Ela descreve as operações de criação, consulta, atualização e exclusão de dados, além dos respectivos caminhos de acesso e códigos de resposta HTTP esperados. Essas rotas permitem a comunicação entre o frontend e o backend, viabilizando o gerenciamento das funcionalidades da aplicação.

USUÁRIOS

ENDPOINTS — USUÁRIOS

EcoGuia • API RESTful

URI	MÉTODO	DESCRIÇÃO	STATUS HTTP
/usuarios	GET	Lista todos os usuários	200 OK / 404 Not Found
/usuarios/{idUsuario}	GET	Consulta um usuário específico	200 OK / 404 Not Found
/usuarios	POST	Cadastra um novo usuário	201 Created / 400 Bad Request
/usuarios/{idUsuario}	PUT	Atualiza os dados de um usuário	201 Created / 400 Bad Request / 404 Not Found
/usuarios/{idUsuario}	DELETE	Exclui um usuário	204 No Content / 404 Not Found

Métodos e códigos de resposta conforme a implementação documentada.

AVATARES

ENDPOINTS — AVATARES

EcoGuia • API RESTful

URI	MÉTODO	DESCRIÇÃO	STATUS HTTP
/avatares	GET	Lista todos os avatares	200 OK / 404 Not Found
/avatares/{idAvatar}	GET	Consulta um avatar específico	200 OK / 404 Not Found
/avatares	POST	Cadastra um novo avatar	201 Created / 400 Bad Request
/avatares/{idAvatar}	PUT	Atualiza os dados de um avatar	201 Created / 400 Bad Request / 404 Not Found
/avatares/{idAvatar}	DELETE	Exclui um avatar	204 No Content / 404 Not Found

Métodos e códigos de resposta conforme a implementação documentada.

BADGES

ENDPOINTS — BADGES

EcoGuia • API RESTful

URI	MÉTODO	DESCRIÇÃO	STATUS HTTP
/badges	GET	Lista todos os badges	200 OK / 404 Not Found
/badges/{idBadge}	GET	Consulta um badge específico	200 OK / 404 Not Found
/badges	POST	Cadastra um novo badge	201 Created / 400 Bad Request
/badges/{idBadge}	PUT	Atualiza os dados de um badge	201 Created / 400 Bad Request / 404 Not Found
/badges/{idBadge}	DELETE	Exclui um badge	204 No Content / 404 Not Found

Métodos e códigos de resposta conforme a implementação documentada.

CONQUISTAS

ENDPOINTS — CONQUISTAS

EcoGuia • API RESTful

URI	MÉTODO	DESCRIÇÃO	STATUS HTTP
/conquistas	GET	Lista todas as conquistas	200 OK / 404 Not Found
/conquistas/{idConquista}	GET	Consulta uma conquista específica	200 OK / 404 Not Found
/conquistas	POST	Cadastra uma nova conquista	201 Created / 400 Bad Request
/conquistas/{idConquista}	PUT	Atualiza os dados de uma conquista	201 Created / 400 Bad Request / 404 Not Found
/conquistas/{idConquista}	DELETE	Exclui uma conquista	204 No Content / 404 Not Found

Métodos e códigos de resposta conforme a implementação documentada.

PROTÓTIPO DO PROJETO

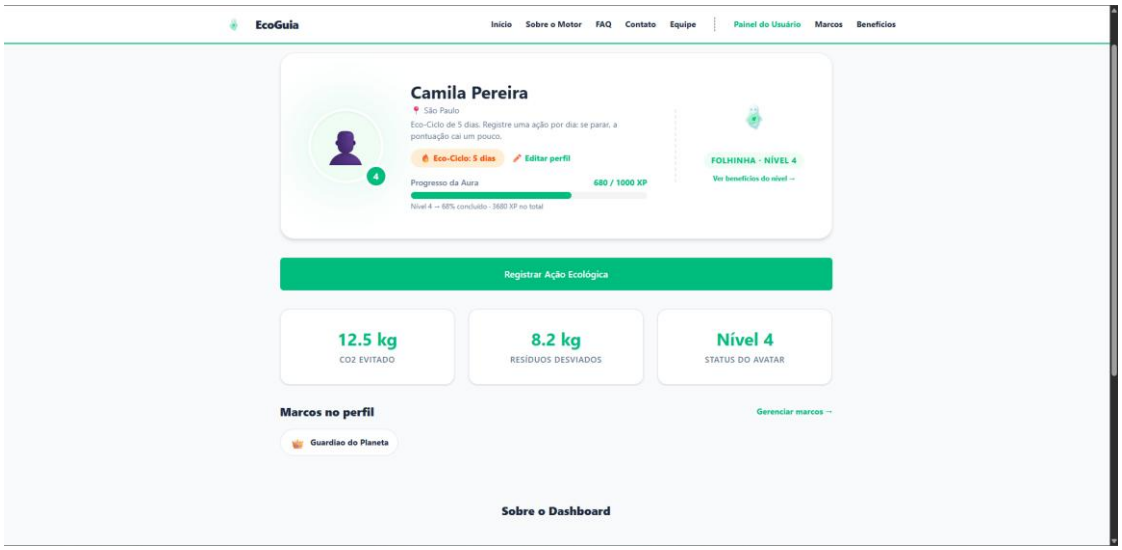
Tela 1 — Página Inicial:



Interação do usuário:

- Visualiza a apresentação do EcoGuia e do motor de reputação
- Navega pelo menu entre as páginas institucionais (Sobre, FAQ, Contato e Equipe) e as páginas do usuário (Painel, Marcos e Benefícios)
- Acessa o Painel do Usuário pelo botão Explorar Dashboard

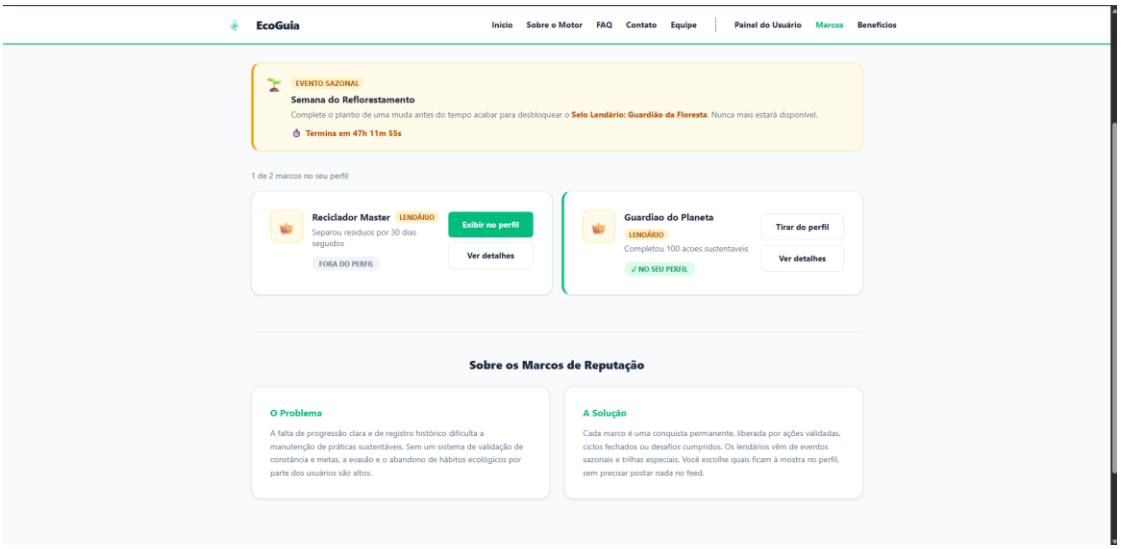
Tela 2 — Painel do Usuário



Interação do usuário:

- Visualiza nome, cidade, nível e XP do avatar, carregados da API (GET /usuarios/1 e GET /avatares/{id})
- Registra uma ação ecológica: o XP é somado e enviado à API (PUT /avatares/{id}), que recalcula o nível
- Acompanha o Eco-Ciclo, a sequência de dias com ação registrada (PUT /usuarios/1)
- Vê os marcos exibidos no perfil (GET /badges e GET /conquistas)
- Acessa a edição do perfil, que altera nome, e-mail e cidade (PUT /usuarios/1)

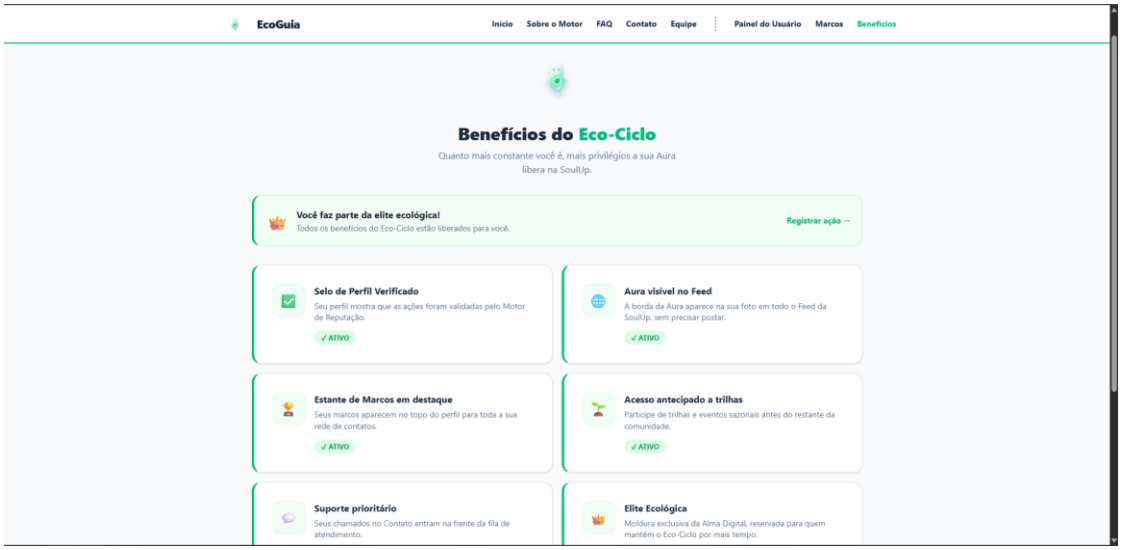
Tela 3 — Marcos de Reputação



Interação do usuário:

- Consulta todos os marcos cadastrados (GET /badges), do tipo normal ou lendário
- Escolhe quais marcos aparecem no perfil: exibir (POST /conquistas) ou tirar do perfil (DELETE /conquistas/{id})
- Abre o detalhe de um marco (GET /badges/{id})

Tela 4 — Benefícios do Eco-Ciclo

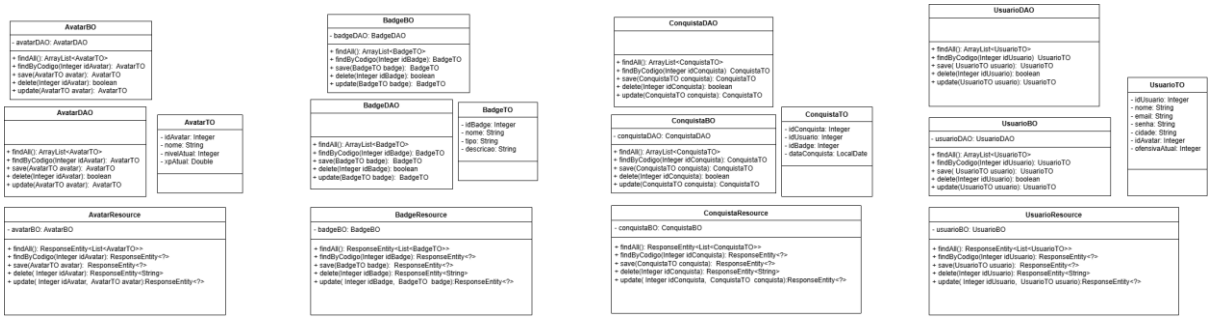


Interação do usuário:

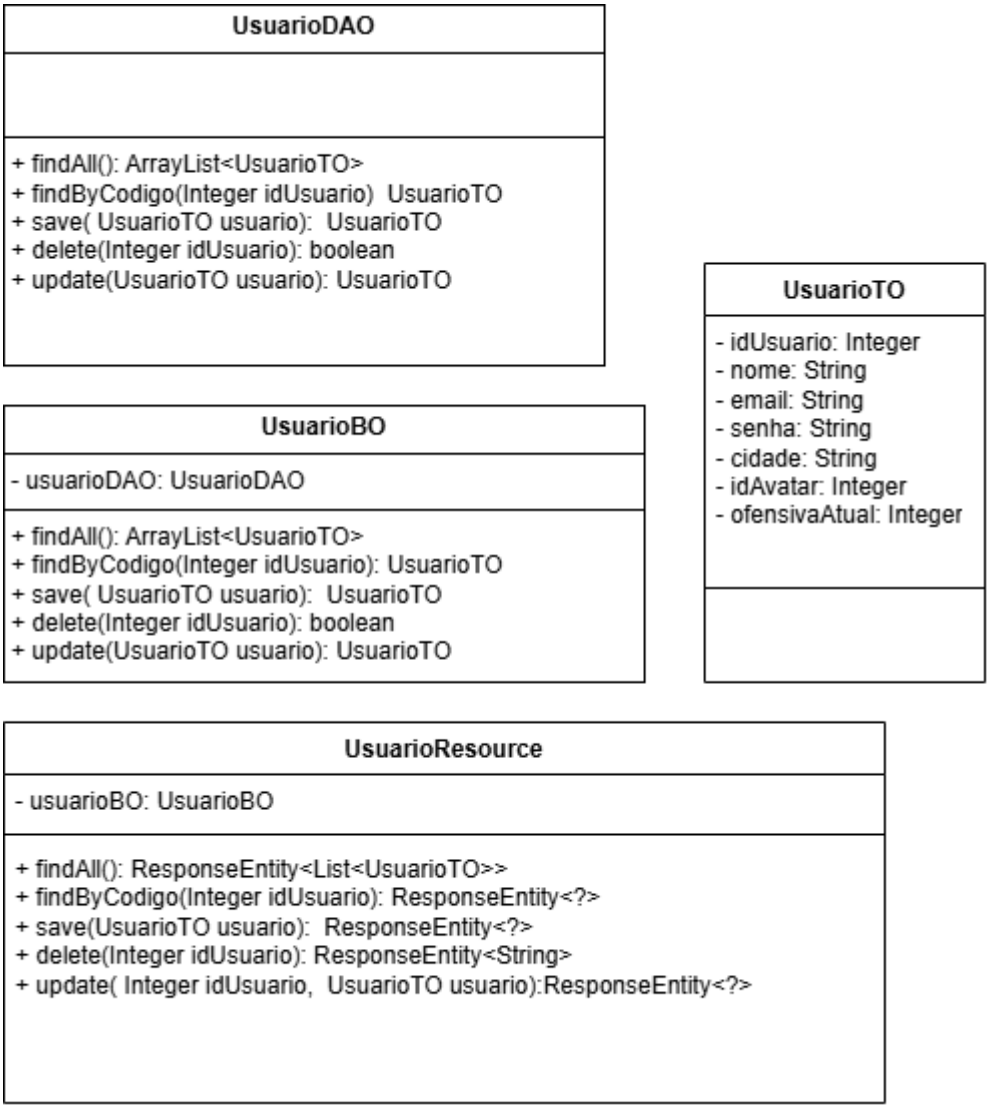
- Visualiza os benefícios do Eco-Ciclo liberados conforme o nível do avatar (GET /avatares/{id})
- Vê quais benefícios já estão ativos e quais ainda faltam desbloquear
- Recebe um aviso com o próximo benefício a liberar ou, quando todos estão liberados, a mensagem de elite ecológica
- Acessa o Painel do Usuário pelo atalho Registrar ação

DIAGRAMA DE CLASSES:

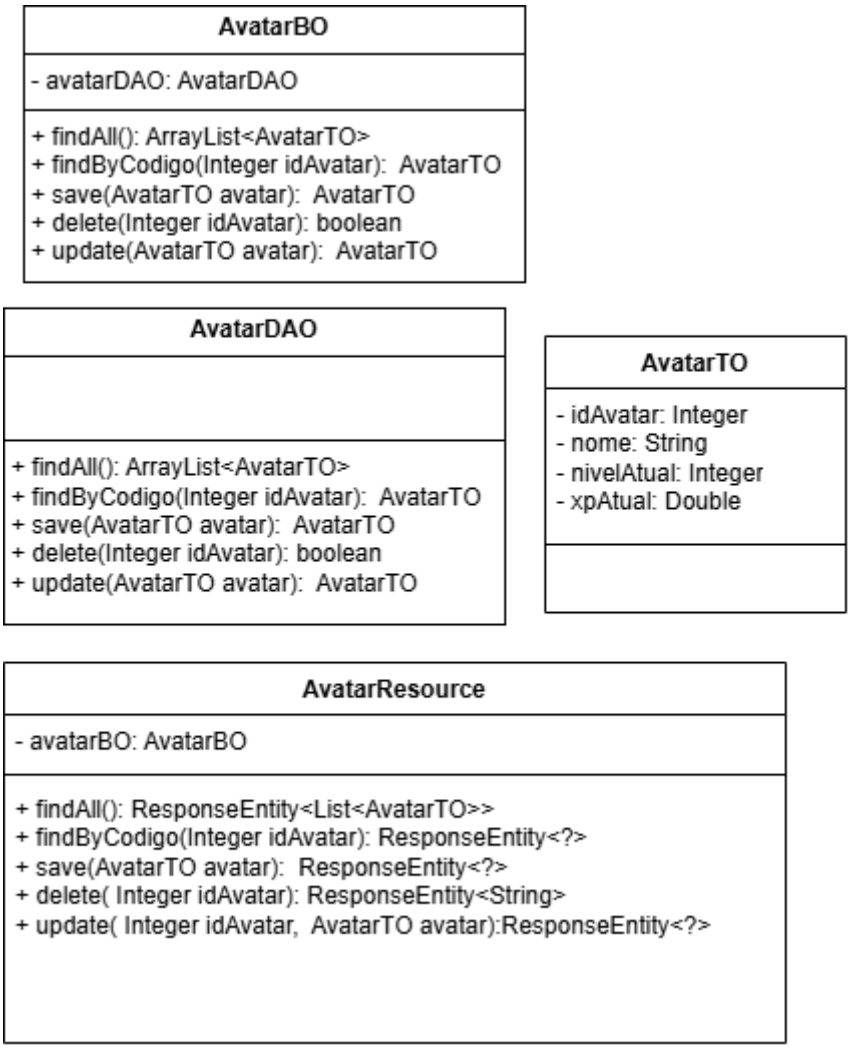
O diagrama de classes geral apresenta a estrutura da aplicação EcoGuia, demonstrando a organização das classes e seus relacionamentos. Nele, estão representadas as camadas de transferência de dados (TO), acesso a dados (DAO), regras de negócio (BO) e recursos REST (Resource), permitindo uma visão ampla da arquitetura do sistema.



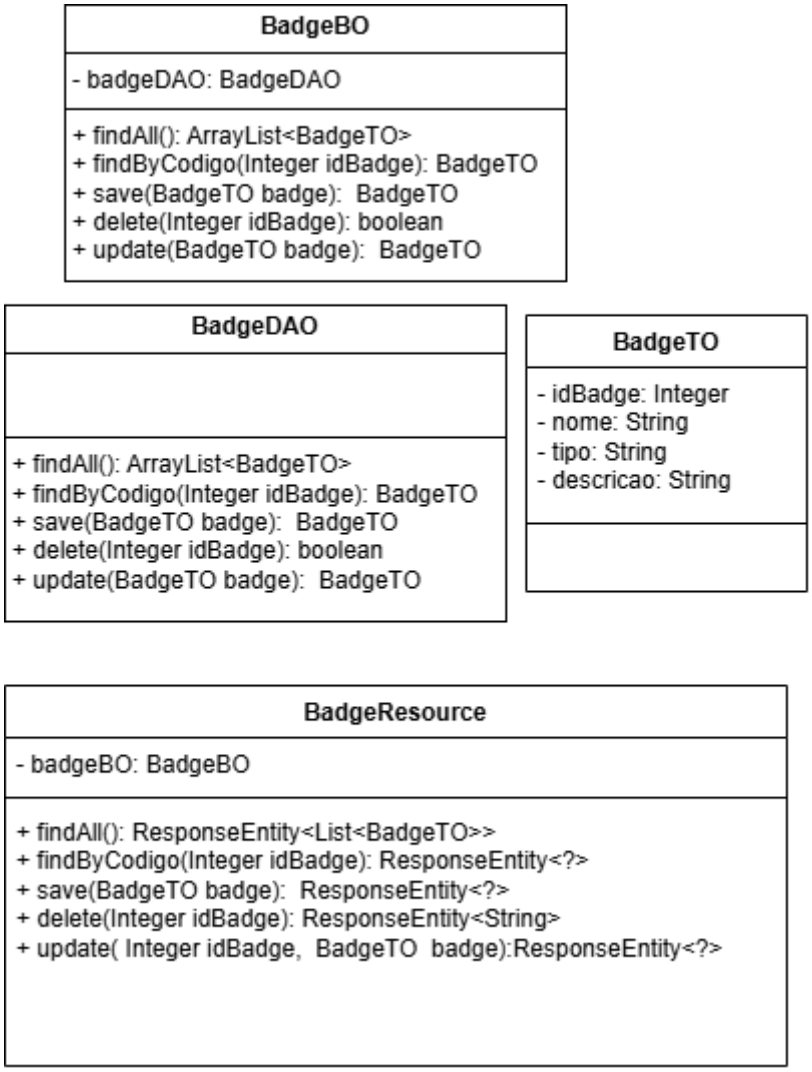
CLASSES USUARIO



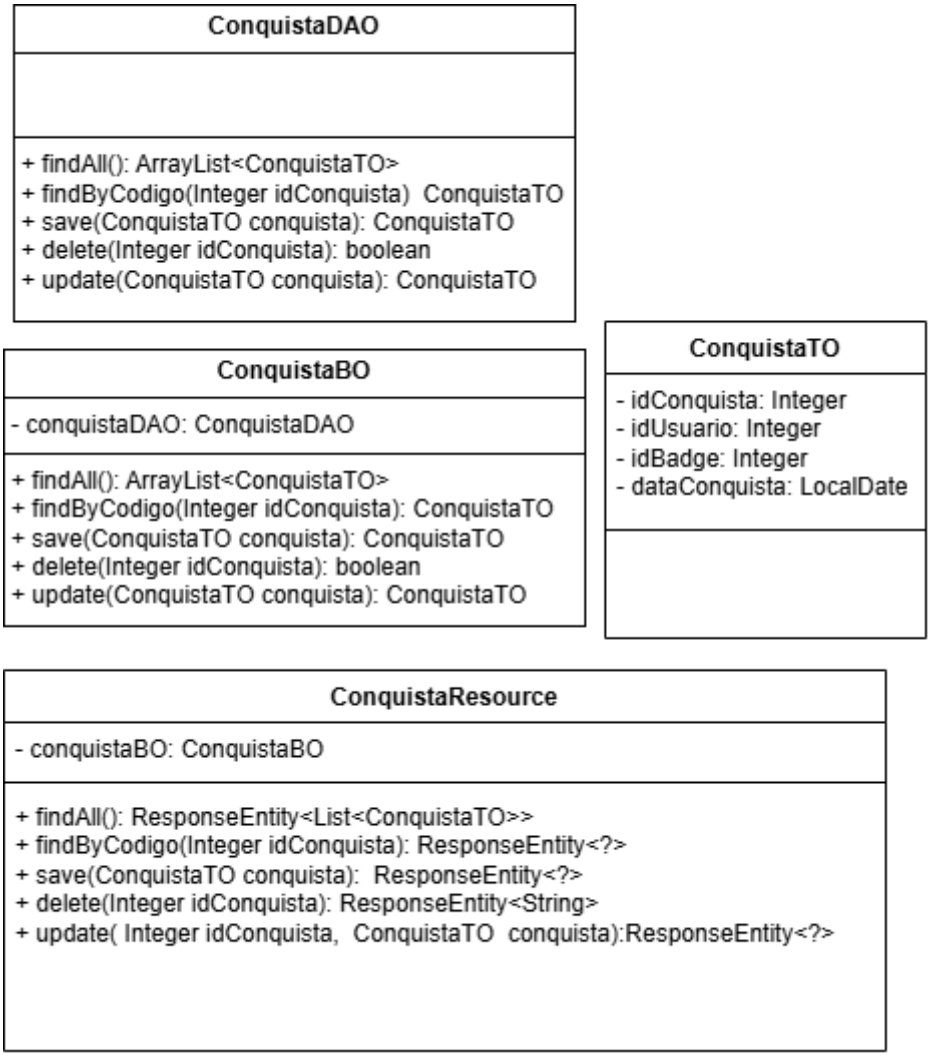
CLASSES AVATAR



CLASSES BADGE



CLASSES CONQUISTA



Procedimentos para rodar a aplicação

1. **IDE:** IntelliJ IDEA
2. **Linguagem:** Java
3. **Framework:** Spring Boot 4.1.2
4. **Banco de dados:** Oracle Database
5. **Driver JDBC:** Oracle JDBC
6. **Gerenciador de dependências:** Maven

7. Versões necessárias

8. JDK 21
9. Maven Wrapper disponibilizado no projeto
10. Oracle Database acessível e configurado
11. Dependências do projeto, gerenciadas pelo Maven

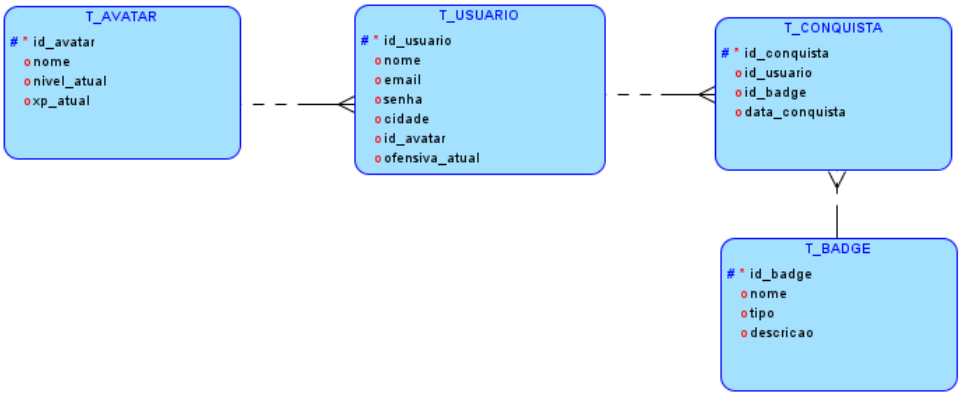
12. Como importar o projeto

13. Abra o IntelliJ IDEA.
14. Selecione **File → Open** e escolha a pasta raiz do projeto.
15. Aguarde o IntelliJ reconhecer e indexar os arquivos.
16. Aguarde a importação das dependências do Maven, utilizando o arquivo `pom.xml`.
17. Verifique se o JDK 21 está configurado em **File → Project Structure → Project → SDK**.

18. Passo a passo da execução

19. Certifique-se de que o banco de dados Oracle está acessível e que as tabelas necessárias para o funcionamento da aplicação foram criadas.
20. Configure as variáveis de ambiente utilizadas na conexão com o banco:
    a. `DB_URL`: URL de conexão com o Oracle.
    b. `DB_USER`: usuário do banco de dados.
    c. `DB_PASSWORD`: senha do banco de dados.
21. Localize a classe principal `EcoguiaApplication`, dentro do pacote `br.com.fiap`.
22. Execute o método `main` dessa classe.
23. Aguarde a inicialização do Spring Boot e a conexão com o banco de dados.
24. Com a aplicação iniciada, a API estará disponível para receber requisições HTTP.
25. As operações podem ser realizadas por meio dos endpoints REST disponibilizados pela aplicação, utilizando métodos HTTP como GET, POST, PUT e DELETE, com envio e recebimento de dados no formato JSON.

Modelo de Entidade-Relacionamento (MER):



SCRIPT DAS TABELAS:

Tabela: t_avatar:

```
CREATE TABLE t_avatar (
   id_avatar   NUMBER GENERATED ALWAYS AS IDENTITY,
   nome        VARCHAR2(30) NOT NULL,
   nivel_atual NUMBER(3)    DEFAULT 1 NOT NULL,
   xp_atual    NUMBER(10,2) DEFAULT 0 NOT NULL,
   CONSTRAINT pk_avatar PRIMARY KEY (id_avatar),
   CONSTRAINT ck_avatar_nivel CHECK (nivel_atual >= 1),
   CONSTRAINT ck_avatar_xp CHECK (xp_atual >= 0)
);
```

Tabela: t_badge:

```
CREATE TABLE t_badge (
   id_badge  NUMBER GENERATED ALWAYS AS IDENTITY,
   nome      VARCHAR2(40)  NOT NULL,
   tipo      VARCHAR2(10)  NOT NULL,
   descricao VARCHAR2(200),
   CONSTRAINT pk_badge PRIMARY KEY (id_badge),
   CONSTRAINT ck_badge_tipo CHECK (tipo IN ('normal', 'lendario')) );
```

Tabela: t_usuario:

```
CREATE TABLE t_usuario (
   id_usuario     NUMBER GENERATED ALWAYS AS IDENTITY,
   nome           VARCHAR2(15)  NOT NULL,
   email          VARCHAR2(100) NOT NULL,
   senha          VARCHAR2(50)  NOT NULL,
   cidade         VARCHAR2(20),
   id_avatar      NUMBER        NOT NULL,
   ofensiva_atual NUMBER(5)     DEFAULT 0 NOT NULL,
   CONSTRAINT pk_usuario PRIMARY KEY (id_usuario),
   CONSTRAINT fk_usuario_avatar FOREIGN KEY (id_avatar)
REFERENCES t_avatar (id_avatar),
   CONSTRAINT ck_usuario_ofensiva CHECK (ofensiva_atual >= 0)
);
```

Tabela: t_conquista:

```
CREATE TABLE t_conquista (
   id_conquista   NUMBER GENERATED ALWAYS AS IDENTITY,
   id_usuario     NUMBER NOT NULL,
   id_badge       NUMBER NOT NULL,
   data_conquista DATE   DEFAULT SYSDATE NOT NULL,
   CONSTRAINT pk_conquista PRIMARY KEY (id_conquista),
   CONSTRAINT fk_conquista_usuario FOREIGN KEY (id_usuario)
REFERENCES t_usuario (id_usuario),
   CONSTRAINT fk_conquista_badge FOREIGN KEY (id_badge)
REFERENCES t_badge (id_badge),
   CONSTRAINT uq_conquista_usuario_badge UNIQUE (id_usuario,
id_badge)
 );
```